

Here is the beautified, unified repository structure. I kept all your descriptions exactly the same, but upgraded the formatting using standard folder tree architecture and clean emojis so it is much easier to read and share with your team.

📦 **edge_guard_workspace** → The single monorepo containing the FastAPI backend, the local guardrail engine, and deployment configs.

|

└─ 📁 **guardrails** → Contains all standalone Python modules for text evaluation, policy enforcement, and SLM communication. (*TRACK A*)

| └─ 📁 **deterministic** → Handles instantaneous, rule-based text sanitization and PII redaction without relying on AI.

| | └─ 📄 **normalizer.py** → Flattens messy text to prevent attackers from sneaking prompts past filters using obfuscation.

| | | └─ ⚙️ *i/p: raw user prompt string → removes markdown, decodes base64, normalizes unicode → o/p: clean plaintext string*

| | └─ 📄 **pii_scrubber.py** → Integrates Microsoft Presidio to detect and mask sensitive user information locally.

| | | └─ ⚙️ *i/p: clean plaintext string → runs local NER and regex analyzers → o/p: redacted string (e.g., "Call [PHONE_NUMBER]")*

| | └─ 📄 **keyword_scanner.py** → Instantly blocks hardcoded banned words or universal jailbreak prefixes.

| | └─ ⚙️ *i/p: redacted string → scans against predefined regex blocklists → o/p: boolean safety status and triggered keyword*

| |

| └─ 📁 **semantic** → Manages the intelligence layer, dynamic prompting, and local GPU SLM execution.

| | └─ 📄 **policy_parser.py** → Translates dynamic user JSON configurations into Llama Guard system instructions.

| | | └─ ⚙️ *i/p: JSON policy dictionary → maps rules to Llama Guard harm categories → o/p: strict system prompt string*

| | └─ 📄 **llama_wrapper.py** → Connects to the local server to execute the 1B Llama Guard model.

| | | └─ ⚙️ *i/p: formatted prompt string → sends async request to local GPU → o/p: raw SLM text response*

| | └─ 📄 **evaluator.py** → Orchestrates the SLM check by combining the parsed policy and

user prompt, then structuring the final verdict.

| | └  *i/p: user prompt & JSON policy → calls policy_parser then llama_wrapper → o/p: strict JSON {"is_safe": bool, "reason": str}*

| |

| └  **schemas** → Defines the strict data structures used to ensure reliable communication between the guardrails and the backend.

| └  **data_models.py** → Contains Pydantic models to ensure consistent data passing between functions.

| └  *i/p: raw dictionary data → validates data types and required fields → o/p: validated Pydantic object*

|

| └  **backend_api** → Contains the FastAPI application, LLM routing logic, and server configurations. (TRACK B)

| └  **routers** → Defines the API endpoints (e.g., /v1/chat/completions) that user applications will call to interact with the gateway.

| └  **middleware** → Handles the interception of inbound/outbound requests, injecting Track A's guardrail evaluations into the traffic flow.

| └  **services** → Manages the connection to target LLMs (via LiteLLM) and handles the buffer logic for streaming/chunked output evaluation.

|

| └  **deployment** → Contains the infrastructure files required to package the entire project as an On-Premise Microservice. (TRACK B)

| └  **Dockerfile** → The script that builds the isolated container environment holding both the API and Track A scripts.

| └  **docker-compose.yml** → Orchestrates the startup of the API container alongside necessary local services like Redis and Ollama.

|

└  **tests** → Shared folder for automated testing scripts across both tracks. (SHARED)

| └  **red_team** → Contains scripts (like Garak/PromptInject) to aggressively test Track A's guardrails with known jailbreaks.

└  **integration** → Contains scripts to ensure Track B's proxy correctly routes and blocks traffic using Track A's logic.